

TP: Introduction to Deep Learning with Keras

MNIST Dataset

Let's look at a real-world example of a neural network. We will use the Keras library to learn how to classify handwritten numbers.

The problem we are considering is to classify handwritten ten-digit grayscale images (28×28 pixels) into their 10 categories (0 to 9).

We will use the MNIST dataset, a classic of the machine learning community.

It is a set of 60,000 training images, plus 10,000 test images, assembled by the National Institute of Standards and Technology.

Here are examples based on numbers 2 and 4:



The MNIST dataset is preloaded into Keras as a set of Numpy arrays.

Loading the MNIST dataset into Keras

1. Based on Keras' documentation, explain what the `train_images`, `train_labels`, `test_images` and `test_labels` data represent and what they will be used for.
2. Give the shape of encoding training images as Numpy arrays
3. Give the shape of the encoding of the labels `train_labels`.
4. Same question with test images and their labels.

The work will be organized as follows: First, we will feed the neural network with training data: training images (`train`) and training labels. The network will then learn how to associate images and labels. Finally, we will ask the network to produce predictions for `test_images`, and we will check if these predictions match the `test_labels` labels.

Building the network

```
from keras import models
```

```
from keras import layers
```

```
network = models.Sequential()
```

```
network.add(layers.Dense(512, activation='relu', input_shape=(28 * 28,)))
```

```
network.add(layers.Dense(10, activation='softmax'))
```

5. Explain the above code, specify the number of layers, the number of neurons per layer and the function of the last layer.

Compiling the model

Definitions:

- A loss function tells us how the network will be able to measure its performance on the training data, and therefore how it will be able to navigate to the right solution.
 - An optimizer is the mechanism by which the network will update itself based on the data it sees and its loss function.
 - The metric is used to monitor results during training and testing.
6. Write the code to give the arguments of the compilation of the model by choosing as loss function "categorical_crossentropy", as optimizer, "rmsprop" and as metric "accuracy" (i.e., the fraction of the images that have been correctly classified.).

Data pre-processing

Before training, we will preprocess the data by reshaping it into the shape that the network expects and scaling it so that all values are in the interval [0, 1].

Previously, our training images were stored in an array of shapes (60000, 28, 28) of type uint8 with values in the interval [0, 255].

7. Turn the array into a float32 (60000, 28*28) form array with values between 0 and 1.

Pre-treatment of labels

8. Use the code below. Explain it.

```
from keras.utils import to_categorical  
train_labels = to_categorical(train_labels)  
test_labels = to_categorical(test_labels)
```

Model training

9. Train the model using the "fit" method with 5 epochs and a lot size of 128. See documentation.

10. Explain the results obtained.

Model Testing

11. Run the model on the test data (images).

12. Conclude.

Not a : RMSprop is a gradient-based optimization technique used in the formation of neural networks. It was proposed by Geoffrey Hinton. The gradients of very complex functions like neural networks tend to "disappear" or explode as data propagates through the function. Rmsprop was developed as a stochastic technique for mini-batch learning. RMSprop addresses the above problem by using a moving average of squared gradients to normalize the gradient. This normalization balances the size of the pitch (momentum), decreasing the pitch for large gradients to avoid exploding and increasing the pitch for small gradients to avoid disappearing.

Simply put, RMSprop uses an adaptive learning rate instead of treating the learning rate as a hyperparameter. This means that the learning rate changes over time.